รายงานโครงงาน

RiceCare AI

เว็บแอปพลิเคชันสำหรับจำแนกโรคใบข้าวด้วยปัญญาประดิษฐ์

จัดทำขึ้นเพื่อศึกษาและพัฒนาระบบต้นแบบด้าน Image Processing และ Deep Learning

บทคัดย่อ

โครงงาน RiceCare AI เป็นการพัฒนาระบบต้นแบบเว็บแอปพลิเคชันสำหรับจำแนกโรคใบข้าวจากภาพถ่าย โดยประยุกต์ใช้เทคนิคการประมวลผลภาพร่วมกับการเรียนรู้เชิงลึก (Deep Learning) เพื่อช่วยประเมินลักษณะของใบข้าวและแสดงผลการจำแนกประเภทโรคผ่านหน้าเว็บไซต์ ระบบแบ่งการทำงานออกเป็น 3 ส่วนหลัก ได้แก่ ส่วนติดต่อผู้ใช้ (Frontend) สำหรับอัปโหลดภาพและแสดงผล ส่วนบริการประมวลผล (Backend) ที่พัฒนาด้วย FastAPI และส่วนแบบจำลองปัญญาประดิษฐ์ที่สร้างด้วย TensorFlow/Keras

แบบจำลองใช้ MobileNetV2 ซึ่งเป็นโครงข่ายประสาทเทียมที่ผ่านการฝึกจาก ImageNet มาแล้ว โดยนำมาใช้ในแนวทาง Transfer Learning และเพิ่มชั้น Global Average Pooling, Dropout และ Dense สำหรับจำแนกภาพออกเป็น 6 กลุ่ม ได้แก่ Bacterial Blight, Brown Spot, Healthy, Leaf Blast, Leaf Smut และ Tungro ชุดข้อมูลที่ใช้แบ่งเป็น Train จำนวน 5,287 ภาพ, Validation จำนวน 1,135 ภาพ และ Test จำนวน 1,141 ภาพ รวม 7,563 ภาพ

ผลการทดสอบที่บันทึกไว้รายงานว่าระบบจำแนกภาพชุดทดสอบได้ถูกต้อง 1,041 จาก 1,141 ภาพ คิดเป็นความแม่นยำ 91.24%
ผลดังกล่าวสะท้อนว่าแบบจำลองสามารถใช้เป็นต้นแบบเพื่อศึกษาการจำแนกโรคใบข้าวได้ อย่างไรก็ตาม ผลลัพธ์จากระบบไม่ควรใช้แทนการวินิจฉัยโดยผู้เชี่ยวชาญทางการเกษตร เนื่องจากภาพถ่าย สภาพแสง ระยะของโรค และความแตกต่างของพันธุ์ข้าวอาจส่งผลต่อการทำนาย

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ข้าวเป็นพืชเศรษฐกิจและเป็นอาหารหลักที่มีความสำคัญต่อประเทศไทยผลผลิตข้าวอาจได้รับผลกระทบจากโรคพืชหลายชนิด ซึ่งบางโรคมีอาการปรากฏบนใบ เช่น จุดสีน้ำตาล รอยแผล หรือการเปลี่ยนสีของเนื้อเยื่อ การสังเกตอาการจากภาพด้วยตนเองอาจทำได้ยากโดยเฉพาะเมื่ออาการของโรคมีลักษณะใกล้เคียงกันหรือเกิดขึ้นในระยะเริ่มต้น

ความก้าวหน้าของปัญญาประดิษฐ์ด้านการมองเห็นของคอมพิวเตอร์ (Computer Vision) ทำให้สามารถนำภาพถ่ายมาใช้ฝึกแบบจำลองเพื่อเรียนรู้รูปแบบที่สัมพันธ์กับประเภทของโรคได้ โครงงาน RiceCare AI จึงมุ่งพัฒนาระบบต้นแบบที่ให้ผู้ใช้ส่งภาพใบข้าวผ่านเว็บไซต์แล้วรับผลการจำแนกพร้อมคะแนนความเชื่อมั่นของแบบจำลอง เพื่อสนับสนุนการคัดกรองเบื้องต้นและเป็นสื่อการเรียนรู้เกี่ยวกับ AI

1.2 วัตถุประสงค์

- เพื่อพัฒนาแบบจำลอง Deep Learning สำหรับจำแนกภาพใบข้าวออกเป็น 6 กลุ่มตามชุดข้อมูลที่ใช้
- เพื่อประยุกต์ใช้ Transfer Learning ด้วย MobileNetV2 เพื่อลดภาระในการฝึกแบบจำลองจากศูนย์
- เพื่อพัฒนาเว็บแอปพลิเคชันที่รับภาพจากผู้ใช้และแสดงผลการทำนายจากแบบจำลอง
- เพื่อประเมินประสิทธิภาพเบื้องต้นด้วยชุดข้อมูลทดสอบที่แยกจากชุดฝึก
- เพื่อศึกษาการเชื่อมต่อระหว่าง Frontend, Backend API และแบบจำลอง AI

1.3 ขอบเขตของโครงงาน

- ระบบรับภาพใบข้าวจากผู้ใช้ผ่านหน้าเว็บไซต์ และส่งภาพไปยัง Backend เพื่อประมวลผล

- ระบบจำแนกภาพตาม 6 คลาส ได้แก่ Bacterial Blight, Brown Spot, Healthy, Leaf Blast, Leaf Smut และ Tungro
- ระบบแสดงคลาสที่ทำนายได้และคะแนนของแต่ละคลาสจากแบบจำลอง
- แบบจำลองที่พัฒนาเป็นต้นแบบเพื่อการศึกษา ไม่ใช่ระบบวินิจฉัยโรคพืชที่ได้รับการรับรอง
- ขอบเขตปัจจุบันไม่รวมการเชื่อมต่อฐานข้อมูลผู้ใช้ ระบบสมาชิก การจัดเก็บประวัติการวิเคราะห์ หรือการให้คำแนะนำการใช้สารเคมี

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- ได้เรียนรู้กระบวนการเตรียมข้อมูลภาพ ฝึกแบบจำลอง และประเมินผลโมเดล AI
- ได้ต้นแบบเว็บแอปพลิเคชันที่เชื่อมต่อการทำนายภาพผ่าน REST API
- ได้ฝึกการออกแบบสถาปัตยกรรมแยก Frontend และ Backend อย่างเป็นระบบ
- ได้แนวทางต่อยอดสู่การทดสอบกับภาพจากสภาพแวดล้อมจริงและการพัฒนาโมเดลให้มีความทนทานมากขึ้น

บทที่ 2 เทคโนโลยีและเครื่องมือ

องค์ประกอบ	เทคโนโลยี	หน้าที่
AI Model	Python, TensorFlow/Keras, MobileNetV2	ฝึกและเรียกใช้แบบจำลองจำแนกภาพใบข้าว
ประมวลผลภาพ	Pillow, NumPy	อ่านภาพ แปลงเป็น RGB ปรับขนาด และเตรียมข้อมูลเป็นอาร์เรย์
Backend API	FastAPI, Uvicorn	รับภาพจาก Frontend เรียกโมเดล และส่งผลลัพธ์เป็น JSON
Frontend	HTML, CSS, JavaScript, Vite	สร้างหน้าเว็บไซต์สำหรับเลือกภาพ แสดงสถานะ และแสดงผลทำนาย
Dataset	Kaggle rice leaf disease dataset	ใช้เป็นข้อมูลภาพสำหรับฝึก ตรวจสอบระหว่างฝึก และทดสอบ
เครื่องมือพัฒนา	Google Colab, VS Code, Web Browser	ฝึกโมเดลและพัฒนา/ทดสอบระบบ

บทที่ 3 ข้อมูลและการพัฒนาแบบจำลอง

3.1 ชุดข้อมูล

โครงงานใช้ชุดข้อมูลภาพใบข้าวจาก Kaggle ที่จัดโครงสร้างเป็นโฟลเดอร์ Train, Valid และ Test เพื่อแยกบทบาทของข้อมูลอย่างชัดเจน โดย Train ใช้ปรับค่าน้ำหนักของแบบจำลอง, Validation ใช้ติดตามผลระหว่างการฝึกและเลือกแบบจำลอง และ Test ใช้ประเมินผลหลังการฝึก

ชุดข้อมูล	จำนวนภาพ	วัตถุประสงค์
Train	5,287	ใช้ฝึกแบบจำลอง
Validation	1,135	ติดตามผลและเลือกโมเดล
Test	1,141	ประเมินผลขั้นสุดท้าย
รวม	7,563	จำนวนภาพทั้งหมด

3.2 ประเภทที่แบบจำลองจำแนก

- Bacterial Blight — โรคขอบใบแห้งจากแบคทีเรีย
- Brown Spot — โรคใบจุดสีน้ำตาล
- Healthy — ใบข้าวที่จัดอยู่ในกลุ่มปกติตามชุดข้อมูล
- Leaf Blast — โรคไหม้
- Leaf Smut — โรคใบขีดหรือโรคใบสมัทตามป้ายกำกับของชุดข้อมูล
- Tungro — โรคข้าวที่เกี่ยวข้องกับเชื้อไวรัส tungro

หมายเหตุ: ชื่อคลาสในระบบยึดตามป้ายกำกับของชุดข้อมูลต้นทาง
การระบุชื่อโรคภาษาไทยควรตรวจสอบกับคำอธิบายของผู้จัดทำชุดข้อมูลหรือผู้เชี่ยวชาญก่อนนำไปใช้เป็นข้อมูลทางการ

3.3 การเตรียมภาพ

ภาพถูกโหลดจากโฟลเดอร์ตามชื่อคลาสและปรับขนาดเป็น 224 × 224 พิกเซล เพื่อให้มีขนาดตรงกับอินพุตของ MobileNetV2 ใช้ batch size เท่ากับ 32 และกำหนดป้ายกำกับเป็นตัวเลขจำนวนเต็มตามคลาส ภายในแบบจำลองมีชั้น Rescaling ที่แปลงค่าพิกเซลจากช่วง 0–255 ไปเป็นช่วงประมาณ -1 ถึง 1 ซึ่งสอดคล้องกับการเตรียมอินพุตของ MobileNetV2

3.4 โครงสร้างแบบจำลอง

แบบจำลองใช้ MobileNetV2 ที่ผ่านการฝึกบน ImageNet เป็นฐานสำหรับดึงคุณลักษณะจากภาพ โดยตรึงน้ำหนักของฐานโมเดลในช่วงการฝึกนี้ จากนั้นนำผลลัพธ์ผ่าน GlobalAveragePooling2D เพื่อลดมิติของคุณลักษณะ, Dropout อัตรา 0.3 เพื่อลดความเสี่ยงของการจำข้อมูลฝึกมากเกินไป และ Dense จำนวน 6 หน่วยพร้อม Softmax เพื่อสร้างคะแนนสำหรับแต่ละคลาส

1. Input: ภาพ RGB ขนาด 224 × 224 × 3
2. Rescaling: ปรับสเกลค่าพิกเซล
3. MobileNetV2: สกัดคุณลักษณะจากภาพด้วยน้ำหนักที่ฝึกไว้ล่วงหน้า
4. GlobalAveragePooling2D: สรุปคุณลักษณะเชิงพื้นที่
5. Dropout(0.3): ลดการพึ่งพาคุณลักษณะบางส่วนมากเกินไป
6. Dense(6, softmax): ให้คะแนนการทำนายสำหรับ 6 คลาส

3.5 การฝึกโมเดล

มีการฝึกแบบจำลองเป็นเวลา 10 epochs และใช้ EarlyStopping เพื่อหยุดการฝึกเมื่อผลบนชุด Validation ไม่ดีขึ้นตามเงื่อนไขที่กำหนด พร้อมคืนค่าน้ำหนักที่ให้ผลดีที่สุดในระหว่างการฝึก ไฟล์โมเดลที่บันทึกจากกระบวนการดังกล่าวคือ best_rice_model.keras

ผลที่บันทึกไว้ในรอบการฝึกแสดงค่า Validation Accuracy สูงสุดประมาณ 89.34% ที่ Epoch 9 และ Validation Loss ประมาณ 0.3241 ค่าดังกล่าวเป็นผลบนชุด Validation ไม่ใช่ผลบนชุด Test

บทที่ 4 การออกแบบและพัฒนาระบบ

4.1 สถาปัตยกรรมระบบ

ระบบออกแบบให้แยกส่วนติดต่อผู้ใช้ออกจากส่วนประมวลผล AI เพื่อให้สามารถพัฒนาและทดสอบแต่ละส่วนได้อิสระ โดย Frontend ส่งภาพผ่าน HTTP ไปยัง Backend API และ Backend ส่งผลลัพธ์กลับมาเป็น JSON

ผู้ใช้

↓

ลือกภาพใบข้าว

Frontend (HTML / CSS / JavaScript)

↓

HTTP POST /predict (multipart/form-data)

Backend API (FastAPI)

↓

ตรวจสอบและเตรียมภาพ RGB ขนาด 224 × 224

AI Model (TensorFlow / MobileNetV2)

↓

ผลคะแนน 6 คลาส

Backend API ส่ง JSON กลับ

↓

Frontend แสดงคลาสที่ทำนายและคะแนน

4.2 ส่วน Frontend

Frontend ทำหน้าที่ให้ผู้ใช้เลือกไฟล์ภาพ แสดงตัวอย่างภาพและสถานะระหว่างส่งคำขอ จากนั้นนำข้อมูล JSON ที่ได้รับจาก Backend มาแสดงชื่อคลาสที่มีคะแนนสูงสุด พร้อมคะแนนของแต่ละคลาส หน้าเว็บเรียก API ผ่าน URL ที่กำหนดด้วยตัวแปรสภาพแวดล้อม VITE_API_URL เพื่อให้เปลี่ยนปลายทาง API ได้โดยไม่ต้องแก้โค้ดหลายตำแหน่ง

4.3 ส่วน Backend

Backend พัฒนาด้วย FastAPI และให้บริการผ่าน Uvicorn โดยโหลดไฟล์ best_rice_model.keras และไฟล์ class_names.json เมื่อเริ่มระบบ มี endpoint สำหรับตรวจสอบสถานะและ endpoint สำหรับรับภาพเพื่อทำนาย ภาพที่ได้รับจะถูกตรวจสอบชนิดไฟล์ อ่านเป็น RGB ปรับขนาดให้ตรงกับอินพุตของโมเดล แล้วส่งเข้าสู่ model.predict() ผลลัพธ์จะถูกแปลงเป็น JSON เพื่อให้ Frontend นำไปแสดงผล

4.4 API ที่ให้บริการ

Method	Endpoint	หน้าที่	ผลลัพธ์
GET	/	ตรวจสอบว่า API ทำงาน	ข้อความสถานะ
GET	/health	ตรวจสอบความพร้อมของบริการ	สถานะบริการ
POST	/predict	รับภาพเพื่อทำนาย	คลาสที่ทำนาย คะแนน และคะแนนรายคลาส

4.5 ตัวอย่างผลลัพธ์ API


```
{
  "predicted_class": "Brown Spot",
  "confidence": 0.91,
  "confidence_percent": 91.0,
  "scores": {
    "Bacterial Blight": 0.02,
    "Brown Spot": 0.91,
    "Healthy": 0.01,
    "Leaf Blast": 0.03,
    "Leaf Smut": 0.01,
    "Tungro": 0.02
  }
}
```


ตัวเลขในตัวอย่างเป็นข้อมูลสมมติที่ใช้แสดงรูปแบบ JSON เท่านั้น ไม่ใช่ผลจากภาพทดสอบจริง คะแนน Softmax แสดงความมั่นใจสัมพัทธ์ของแบบจำลอง ไม่ได้รับประกันว่าเป็นความน่าจะเป็นที่ผ่านการปรับเทียบแล้ว

4.6 ขั้นตอนการทำงาน

7. ผู้ใช้เปิดหน้าเว็บ RiceCare AI และเลือกภาพใบข้าว
8. Frontend ตรวจสอบว่ามีไฟล์ภาพที่เลือก แล้วส่งภาพไปยัง endpoint /predict
9. Backend รับไฟล์ ตรวจสอบและแปลงภาพเป็น RGB ขนาด 224 × 224
10. แบบจำลองประมวลผลภาพและสร้างคะแนนสำหรับทั้ง 6 คลาส
11. Backend เลือกคลาสที่มีคะแนนสูงสุดและส่งผลลัพธ์กลับเป็น JSON
12. Frontend แสดงผลการทำนายและคะแนน พร้อมแจ้งว่าผลลัพธ์เป็นเพียงการคัดกรองเบื้องต้น

บทที่ 5 ผลการทดลองและการประเมิน

5.1 ผลการฝึก

จากบันทึกการฝึก แบบจำลองให้ค่า Validation Accuracy สูงสุดประมาณ 89.34% และ Validation Loss ประมาณ 0.3241 ที่ Epoch 9 จึงใช้แนวทาง EarlyStopping เพื่อเลือกน้ำหนักที่มีผลบนชุด Validation ดีที่สุดแทนการใช้ epoch สุดท้ายโดยอัตโนมัติ

5.2 ผลการทดสอบ

ผลการประเมินบนชุด Test ที่บันทึกไว้มีภาพทั้งหมด 1,141 ภาพ และจำแนกถูกต้อง 1,041 ภาพ คิดเป็น Test Accuracy 91.24% โดยคำนวณจากจำนวนภาพที่จำแนกถูกต้องหารด้วยจำนวนภาพทดสอบทั้งหมด

คลาส	ทำนายถูก / จำนวนภาพ	อัตราถูกโดยประมาณ
Bacterial Blight	233 / 259	89.96%
Brown Spot	193 / 205	94.15%
Healthy	255 / 263	96.96%
Leaf Blast	145 / 188	77.13%
Leaf Smut	21 / 29	72.41%
Tungro	194 / 197	98.48%
รวม	1,041 / 1,141	91.24%

ข้อสังเกตจากผลรายคลาส: Tungro และ Healthy มีอัตราจำแนกถูกสูงในชุดทดสอบนี้ ขณะที่ Leaf Blast และ Leaf Smut มีอัตราต่ำกว่าคลาสอื่น โดยเฉพาะ Leaf Smut มีจำนวนตัวอย่างเพียง 29 ภาพ จึงควรตีความผลด้วยความระมัดระวังและเพิ่มตัวอย่างให้สมดุลก่อนนำไปใช้งานจริง

หมายเหตุด้านความถูกต้องของรายงาน: ตารางรายคลาสนี้ถอดจากผล confusion matrix ที่บันทึกไว้ก่อนหน้า และผลรวมสอดคล้องกับ 1,041/1,141 ภาพ หากมีการเปลี่ยนโมเดลหรือชุด Test ควรรันประเมินใหม่และอัปเดตตัวเลขทั้งหมดให้ตรงกับผลจริง

5.3 แนวทางทดสอบระบบ

กรณีทดสอบ	ผลที่คาดหวัง	เกณฑ์ผ่าน
เปิดหน้าเว็บ	หน้าเว็บโหลดและแสดงส่วนอัปโหลดภาพ	ไม่พบข้อผิดพลาดในการโหลด
เรียก GET /health	ได้รับสถานะบริการ	ตอบกลับสำเร็จ
อัปโหลดภาพใบข้าวที่รองรับ	ได้รับผลทำนายและคะแนนรายคลาส	มี predicted_class และ scores
อัปโหลดไฟล์ที่ไม่ใช่ภาพ	ระบบแจ้งข้อผิดพลาดที่เข้าใจได้	ไม่ทำให้บริการหยุดทำงาน
ส่งคำขอโดยไม่แนบไฟล์	ระบบปฏิเสธคำขออย่างเหมาะสม	แจ้งว่าไฟล์จำเป็น

ส่งภาพขนาดใหญ่เกินข้อกำหนด	ระบบปฏิเสธไฟล์ตามข้อจำกัด	ไม่ประมวลผลไฟล์เกินขนาด

ตารางนี้เป็นกรณีทดสอบที่กำหนดไว้สำหรับระบบ ไม่ได้กล่าวอ้างว่าทุกกรณีผ่านการทดสอบแล้ว ผลการทดสอบเชิงระบบควรบันทึกหลังทดลองจริงในสภาพแวดล้อมที่ใช้งาน

บทที่ 6 ข้อจำกัดและแนวทางพัฒนาต่อ

6.1 ข้อจำกัด

- ความแม่นยำขึ้นอยู่กับความหลากหลายและคุณภาพของชุดข้อมูลที่ใช้ฝึก
- ภาพจากแปลงจริงอาจมีแสง เงา ฉากหลัง ระยะถ่าย และอาการหลายชนิดแตกต่างจากภาพในชุดข้อมูล
- จำนวนตัวอย่างของบางคลาสไม่สมดุล โดยเฉพาะ Leaf Smut ในชุดทดสอบ
- คะแนนจาก Softmax ไม่ใช่การยืนยันความถูกต้องทางการเกษตร และโมเดลอาจทำนายผิดได้
- ระบบปัจจุบันจำแนกตาม 6 คลาสที่กำหนด
  ไม่สามารถรับประกันว่าจะจำแนกโรคหรือความผิดปกติที่อยู่นอกชุดข้อมูลได้

6.2 แนวทางพัฒนาต่อ

- เพิ่มภาพใบข้าวจากแปลงจริงและเพิ่มจำนวนข้อมูลในคลาสที่มีตัวอย่างน้อย
- ทดลอง Fine-tuning บางชั้นของ MobileNetV2 และเปรียบเทียบกับโมเดลอื่น เช่น EfficientNetB0
- รายงาน Precision, Recall, F1-score และ Confusion Matrix ควบคู่กับ Accuracy
- เพิ่มการแสดงคำเตือนเมื่อคะแนนต่ำ และแนะนำให้ตรวจสอบกับผู้เชี่ยวชาญเมื่อผลไม่ชัดเจน
- เพิ่มระบบจัดเก็บประวัติการวิเคราะห์เมื่อมีความต้องการใช้งานจริง พร้อมออกแบบการคุ้มครองข้อมูล
- ทดสอบความปลอดภัยของ API จำกัดชนิดและขนาดไฟล์ และกำหนด CORS
  ให้ตรงกับโดเมนที่อนุญาตก่อนเผยแพร่
- ทดสอบกับผู้ใช้จริงเพื่อประเมินความเข้าใจง่าย ความเร็ว และความเหมาะสมของการแสดงผล

บทที่ 7 สรุปผล

โครงงาน RiceCare AI ได้พัฒนาต้นแบบระบบจำแนกโรคใบข้าวจากภาพโดยใช้ MobileNetV2 และ TensorFlow/Keras ร่วมกับเว็บแอปพลิเคชันที่แยก Frontend และ Backend อย่างชัดเจน ผู้ใช้สามารถส่งภาพผ่านเว็บไซต์ให้ FastAPI ประมวลผลและรับผลการทำนายกลับมาได้ แนวทางดังกล่าวแสดงให้เห็นการนำ AI ไปเชื่อมต่อกับระบบซอฟต์แวร์จริงตั้งแต่การเตรียมข้อมูล การฝึกโมเดล การออกแบบ API ไปจนถึงการแสดงผล

จากผลที่บันทึกไว้ แบบจำลองมี Test Accuracy 91.24% อย่างไรก็ตาม ความแม่นยำโดยรวมเพียงค่าเดียวไม่เพียงพอสำหรับตัดสินความพร้อมใช้งานจริงเพราะประสิทธิภาพแตกต่างกันในแต่ละคลาสและข้อมูลทดสอบอาจไม่ครอบคลุมสภาพแวดล้อมจริงโครงงานจึงควรได้รับการทดสอบเพิ่มเติมและระบุอย่างชัดเจนว่าเป็นเครื่องมือคัดกรองเบื้องต้น ไม่ใช่เครื่องมือยืนยันการวินิจฉัยโรค

เอกสารอ้างอิง

13. TensorFlow. (ม.ป.ป.). Transfer learning and fine-tuning.
    https://www.tensorflow.org/tutorials/images/transfer_learning
14. TensorFlow. (ม.ป.ป.). MobileNet models.
    https://www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2
15. FastAPI. (ม.ป.ป.). FastAPI documentation. https://fastapi.tiangolo.com/
16. Vite. (ม.ป.ป.). Vite guide. https://vite.dev/guide/
17. Kaggle. (ม.ป.ป.). Rice leaf disease image dataset used in this project.
    https://www.kaggle.com/datasets/sham69/rice-leaf-disease-detection-dataset

หมายเหตุ: ชื่อชุดข้อมูลและ URL ในเอกสารอ้างอิงให้ยึดตามหน้า Kaggle ที่ดาวน์โหลดมาใช้จริง รวมถึงตรวจสอบเงื่อนไขการอนุญาตใช้งานชุดข้อมูลก่อนเผยแพร่ผลงาน

ภาคผนวก ก โครงสร้างไฟล์ระบบ

```
RiceCare-AI/
├──── backend/
│    ├──── app.py
│    ├──── requirements.txt
│    └──── models/
│         ├──── best_rice_model.keras
│         └──── class_names.json
└──── frontend/
     ├──── index.html
     ├──── package.json
     ├──── .env
     └──── src/
          ├──── main.js
          └──── style.css
```

โครงสร้างข้างต้นเป็นโครงสร้างอ้างอิงสำหรับแยกส่วน Frontend และ Backend ไฟล์จริงอาจแตกต่างตามตำแหน่งที่จัดเก็บโมเดลและการตั้งค่าในเครื่องพัฒนา

ภาคผนวก ข คำศัพท์สำคัญ

คำศัพท์	ความหมาย
AI (Artificial Intelligence)	ปัญญาประดิษฐ์ที่ทำให้คอมพิวเตอร์ทำงานที่ต้องอาศัยความสามารถคล้ายการเรียนรู้หรือการตัดสินใจ
Deep Learning	วิธีการเรียนรู้ของเครื่องที่ใช้โครงข่ายประสาทเทียมหลายชั้น
Transfer Learning	การนำโมเดลที่ฝึกไว้แล้วมาใช้เป็นฐานสำหรับงานใหม่
CNN	โครงข่ายประสาทเทียมที่นิยมใช้กับข้อมูลภาพ
API	ช่องทางมาตรฐานให้ซอฟต์แวร์ต่างส่วนสื่อสารกัน
Accuracy	สัดส่วนตัวอย่างที่ทำนายถูกจากตัวอย่างทั้งหมด
Softmax	ฟังก์ชันที่แปลงค่าผลลัพธ์ให้เป็นคะแนนรวมกันประมาณ 1 สำหรับหลายคลาส